AI Security Findings

Overall risk: CRITICAL | Total findings: 17

Executive summary

Risk overview

Total findings: 17. Critical: 4, High: 10, Medium: 3, Low: 0.

Technical summary

Overall risk level for the project: Critical

Key issues to fix first:
1. Command Injection Vulnerabilities – Multiple endpoints directly pass untrusted user input (e.g., via os.popen) into shell commands without proper validation or sanitization, exposing the system to arbitrary code execution.
2. SQL Injection Vulnerabilities – Several functions directly concatenate user input into SQL queries without parameterization, leaving the database vulnerable to manipulation and unauthorized access.
3. Hardcoded Secrets and Credentials – Critical information such as API secrets, JWT secrets, database credentials, and tokens are hardcoded in source code and configuration files. This practice increases the risk of credential exposure and complicates key rotation.
4. Insecure Token Verification – Weak token verification logic, including overly simplistic string comparisons that allow attackers to bypass authentication, substantially lowers the integrity of access controls.
5. Exposure of Sensitive Data – Debug mode enabled in production, insecure configuration endpoints, and logging of sensitive information (e.g., raw passwords) create multiple channels for data leakage.

Quick wins and hardening recommendations:
• Immediately disable debug

AI narrative (LangChain)

Below is the high-level summary of the security scan results:

1. Command Injection Vulnerability (Critical)
- The /run endpoint executes user input without proper sanitization, making it possible for attackers to run arbitrary system commands.

2. SQL Injection Vulnerability (Critical)
- SQL queries in endpoints like /user are built using unsanitized string concatenation, exposing the application to SQL injection attacks.

3. Debug Mode Enabled in Production (Medium)
- Debug mode is active in production, which can reveal sensitive information if accessed by an attacker.

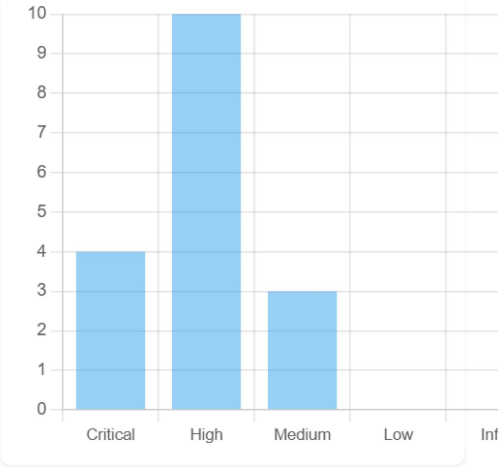
4. Hardcoded API Secret (Medium)
- The presence of hardcoded secrets increases the likelihood of secret leakage if the code becomes accessible.

5. Insecure Token Verification (Medium)
- The authentication flow (for example in the /login endpoint) uses insecure token verification, potentially allowing unauthorized access.

Recommended Next Steps:
- Sanitize and validate all user inputs, especially where system commands are executed.
- Use parameterized queries or an ORM to build SQL queries safely.
- Disable debug mode in production environments.
- Manage secrets securely by using environment variables or dedicated secret management tools.
- Review and strengthen existing token verification mechanisms, ensuring robust authentication practices.

Addressing these issues will significantly improve your project's security posture.

Severity distribution



Severity	Title	File / Location	Description	Recommendation
MEDIUM	Debug Mode Enabled in Production	`app.config["DEBUG"] = True`	Enabling debug mode in production can leak sensitive information through error messages and stack traces. Attackers could exploit this information to further compromise the system.	Set DEBUG to False in production environments and use environment-specific configurations to avoid exposing sensitive details.
HIGH	Hard-Coded Secret	`API_SECRET variable`	The API secret is hard-coded directly in the source code. This presents a significant security risk if the code is ever exposed or shared.	Store secrets in secure environments (such as environment variables or dedicated secret management solutions) that are not checked into source code.
CRITICAL	Command Injection Vulnerability	`run() function, os.popen(cmd).read()`	User input from the 'cmd' query parameter is passed directly to os.popen without proper validation or sanitization. This can lead to arbitrary command execution on the server.	Avoid using os.popen with untrusted input. Validate and sanitize the user input or remove such functionality. Consider using safer alternatives and limiting commands to a whitelist if necessary.
CRITICAL	SQL Injection Vulnerability	`user() function, get_user_raw_query(user_id)`	User input from the 'id' query parameter is passed directly to a function that likely constructs a raw SQL query. If this input is concatenated into the SQL query without proper sanitization or parameterization, it could allow an attacker to manipulate the query.	Use parameterized queries or prepared statements to handle user input in SQL queries. If possible, validate the 'id' parameter to ensure it conforms to the expected format (e.g., numeric).

Severity	Title	File / Location	Description	Recommendation
MEDIUM	Insecure Token Verification	`login() function, verify_token_insecure(token)`	The token verification function, verify_token_insecure, is flagged as insecure. This may indicate weak authentication practices that could be exploited.	Review and strengthen the token verification mechanism. Ensure it uses secure cryptographic practices and proper token validation to prevent unauthorized access.
HIGH	Sensitive Configuration Exposure	`get_config() function`	The endpoint for retrieving configuration data reads and returns a JSON file that may contain sensitive settings. Exposing this configuration can lead to information disclosure, including potential credentials or other sensitive information.	Restrict access to sensitive configurations. Remove or protect endpoints that expose configuration files, and use access controls to ensure only authorized users can view this data.
HIGH	Hardcoded Tokens	`HARDCODED_TOKENS`	The tokens are hardcoded in the source code. This makes it easier for an attacker to discover valid credentials, increases the risk of token leakage, and complicates key rotation.	Store tokens in a secure configuration store or secrets management system instead of hardcoding them. Ensure proper key rotation and access controls.
HIGH	Insecure Token Verification Logic	`verify_token_insecure function`	The token verification accepts any token starting with 'test-', allowing attackers to bypass authentication by providing a token with this prefix. This weakness can be exploited to gain unauthorized access.	Remove generic prefix checks and implement robust, cryptographically secure token validation. Use standard authentication libraries/protocols and avoid simple string comparisons for token validation.
HIGH	Hardcoded Database Credentials	`DB_HOST, DB_USER, DB_PASSWORD in configuration file`	The configuration file contains hardcoded database connection information, including the username ('root') and password ('rootpassword'). Hardcoding credentials in source files can lead to unintended exposure of sensitive data, especially if the code is stored in a version control system or shared with unauthorized users.	Remove hardcoded credentials from configuration files and use a secure secrets management solution (such as environment variables, vaults, or dedicated secrets management tools). Ensure that credentials are rotated regularly.

Severity	Title	File / Location	Description	Recommendation
HIGH	Hardcoded JWT Secret	`JWT_SECRET in configuration file`	The configuration file contains a hardcoded JWT secret which is critical for token signing and verification. If an attacker obtains this secret, they can potentially forge valid tokens, leading to unauthorized access and data compromise.	Do not hardcode the JWT secret in the source code. Use environment variables or a secure secrets management tool to manage sensitive configuration data. Additionally, ensure that the secret is sufficiently long and random.
HIGH	SQL Injection	`get_user function`	User input (uid) is directly concatenated into the SQL query string without proper sanitization or parameterization. This can allow an attacker to inject malicious SQL code, exposing the database to manipulation or unauthorized data access.	Use parameterized queries to safely include user input. For example, instead of concatenating the uid, use a question mark placeholder ('?') and pass uid as a parameter: conn.execute('SELECT * FROM users WHERE id = ?', (uid,))
HIGH	SQL Injection Vulnerability	`get_user_raw_query function, line constructing 'query'`	The function concatenates user-controlled input directly into the SQL query string, making it vulnerable to SQL injection attacks.	Use parameterized queries or prepared statements to safely include user input. For example, use 'cur.execute(query, (user_id,))' with the query string defined as 'SELECT id, username, email FROM users WHERE id = ?'.
MEDIUM	Hardcoded Secret API Key	`Global scope near API_KEY variable`	The secret API key is hardcoded directly in the source code. This can lead to exposure of sensitive credentials if the code is leaked or accessed by unauthorized parties.	Store secrets securely using environment variables or secret management tools, and never hardcode them in the source code.
CRITICAL	Command Injection Vulnerability	`run() function in the /run route`	The 'cmd' parameter from the HTTP request query is passed directly to os.popen(), allowing for potential command injection. An attacker might execute arbitrary shell commands on the host.	Avoid using os.popen() with untrusted input. Validate and sanitize command parameters thoroughly or, if possible, use secure abstractions that do not execute raw shell commands. Consider using subprocess with a list of arguments, and explicitly specify allowed commands.
CRITICAL	SQL Injection Vulnerability	`user() function in the /user route`	User input is directly concatenated into the SQL query string using an f-string. This makes the application susceptible to SQL injection attacks where an attacker	Use parameterized queries or prepared statements to safely include user input in SQL queries. This will prevent SQL injection by properly escaping user-supplied data.

Severity	Title	File / Location	Description	Recommendation
			can modify the intended SQL command.	
HIGH	Cross-Site Scripting (XSS) via DOM Manipulation	`document.getElementById('welcome').innerHTML assignment`	The code uses the URL parameter 'name' directly in the innerHTML property, allowing an attacker to inject malicious scripts (e.g., <script>alert(1)</script>). This constitutes a high severity XSS vulnerability.	Avoid directly inserting untrusted input into the DOM with innerHTML. Use textContent to set content safely, or perform proper sanitization if HTML formatting is needed. For example: document.getElementById('welcome').textContent = 'Hello, ' + name + '!';
HIGH	Logging of Sensitive Information	`log_login_attempt function, logger.warning(f"Login attempt: user={username}, password={password}")`	The function logs raw password information along with the username. This poses a serious security risk as sensitive credentials are being recorded in the logs. If an attacker gains access to the logs, they may retrieve user credentials, leading to credential compromise.	Remove logging of sensitive information such as passwords. Consider logging only non-sensitive details, and implement secure logging practices. If necessary, mask or hash sensitive data before logging.

). This constitutes a high severity XSS vulnerability.", "recommendation": "Avoid directly inserting untrusted input into the DOM with innerHTML. Use textContent to set content safely, or perform proper sanitization if HTML formatting is needed. For example: document.getElementById('welcome').textContent = 'Hello, ' + name + '!';", "source_file": "C:\\Users\\930140\\AppData\\Local\\Temp\\1\\tmpkkecq0gr\\src\\test_project\\static\\app.js" }, { "title": "Logging of Sensitive Information", "severity": "high", "location": "log_login_attempt function, logger.warning(f\"Login attempt: user={username}, password={password}\")", "description": "The function logs raw password information along with the username. This poses a serious security risk as sensitive credentials are being recorded in the logs. If an attacker gains access to the logs, they may retrieve user credentials, leading to credential compromise.", "recommendation": "Remove logging of sensitive information such as passwords. Consider logging only non-sensitive details, and implement secure logging practices. If necessary, mask or hash sensitive data before logging.", "source_file": "C:\\Users\\930140\\AppData\\Local\\Temp\\1\\tmpkkecq0gr\\src\\test_project\\utils\\logger.py" }], "lc_summary": "Below is the high-level summary of the security scan results:\n\n1. Command Injection Vulnerability (Critical)\n - The /run endpoint executes user input without proper sanitization, making it possible for attackers to run arbitrary system commands.\n\n2. SQL Injection Vulnerability (Critical)\n - SQL queries in endpoints like /user are built using unsanitized string concatenation, exposing the application to SQL injection attacks.\n\n3. Debug Mode Enabled in Production (Medium)\n - Debug mode is active in production, which can reveal sensitive information if accessed by an attacker.\n\n4. Hardcoded API Secret (Medium)\n - The presence of hardcoded secrets increases the likelihood of secret leakage if the code becomes accessible.\n\n5. Insecure Token Verification (Medium)\n - The authentication flow (for example in the /login endpoint) uses insecure token verification, potentially allowing unauthorized access.\n\nRecommended Next Steps:\n- Sanitize and validate all user inputs, especially where system commands are executed.\n- Use parameterized queries or an ORM to build SQL queries safely.\n- Disable debug mode in production environments.\n- Manage secrets securely by using environment variables or dedicated secret management tools.\n- Review and strengthen existing token verification mechanisms, ensuring robust authentication practices.\n\nAddressing these issues will significantly improve your project's security posture." }; function downloadJson() { const data = JSON.stringify(reportJson, null, 2); const blob = new Blob([data], { type: "application/json" }); const url = URL.createObjectURL(blob); const a = document.createElement("a"); a.href = url; a.download = "security_report.json"; a.click(); URL.revokeObjectURL(url); }